

AKIRA, as implemented

What the AKIRA repository actually does, traced through entry points, calls, subscriptions, persistence and the model boundary. It is written for the architects who will design VAJRA, and it describes the current code only. Where the code could not settle a question, the question is marked **UNKNOWN**.

branch genesis/foundation-stabilization @ e58020f 581 non-test source files method: 5 parallel tracers + direct verification
no files modified

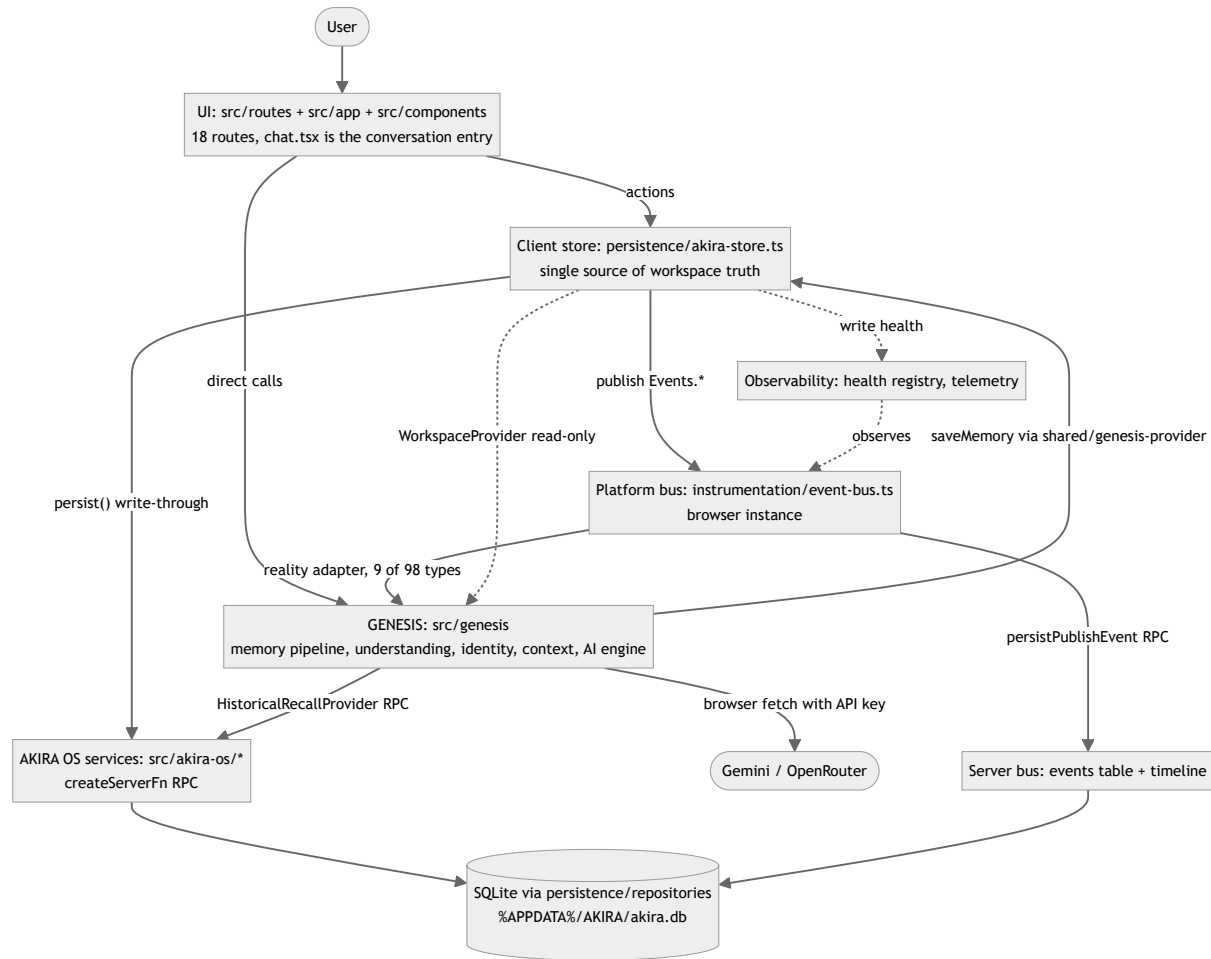
live reached from production inert reached, but has no effect or consumer unreachable no production importer

PART 1

AKIRA at the highest level

AKIRA is a single-user desktop companion built on TanStack Start: React routes, server functions, and a local SQLite database through `better-sqlite3`. Three facts about the runtime come before everything else.

1. **Almost all intelligence runs in the browser.** The client store (`persistence/akira-store.ts`), the whole of GENESIS cognition, context assembly and the LLM calls all execute in the page. The server holds SQLite, the server-function handlers and a second event-bus instance.
2. **The database is the only durable layer.** The workspace is stored as rows. Cognition is stored as one JSON event stream (`settings.genesis_memories`) that is replayed into in-memory state on every start.
3. **The model is a text service, not an agent.** Every decision about what the model sees is deterministic code. Its output is displayed and saved, and is never parsed for actions.



Layers and their directories

Layer	Directories	Size	Responsibility	Kind
Presentation	src/routes , src/app , src/components , src/hooks	119 files, ~19k lines	Pages, shell, the chat conversation loop, the vault UI. Several routes hold real logic (Part 18).	Domain UI

Client state	<code>src/persistence/akira-store.ts</code>	1 file, ~1.4k lines	The workspace state, write-through persistence, event publication, and the GENESIS store provider.	Foundational, coordination
AKIRA OS services	<code>src/akira-os/*</code>	42 files	One module per domain (projects, tasks, notes, sessions, settings, conversations, search, timeline, vault, presence, tools). Each pairs a <code>server/</code> RPC with a <code>services/</code> client wrapper.	Domain, execution
Persistence	<code>src/persistence</code> (connection, initializer, schema.sql, repositories, store-init, reset, migration)	25 files	SQLite schema, migrations, repositories, hydration, reset.	Infrastructure, persists state
Instrumentation	<code>src/instrumentation</code>	17 files	The platform event bus (two instances), the publisher, client-to-server event RPC, the events table and the timeline subscriber.	Infrastructure, communication
GENESIS	<code>src/genesis</code>	207 files, ~25k lines	Memory pipeline, understanding, identity, insights, session/companion context, context assembly, model dispatch.	Cognitive
Contracts / seams	<code>src/contracts</code> , <code>src/shared</code>	26 files	Event names, WorkspaceProvider, HistoricalRecallProvider, the presence channel, repository interfaces, <code>shared/genesis-provider</code> .	Foundational
Observability	<code>src/observability</code>	40 files (21 reachable)	Health registry, telemetry store, event-bus delivery observer. Records data that nothing reads.	Infrastructure
Platform runtime (designed)	<code>src/runtime</code> , <code>src/compatibility</code> , <code>src/sdk</code> , <code>src/analytics</code>	97 files	Module lifecycle, capability registry, SDK, analytics. Accepted in ADR-016..020, but no production importer.	unreachable

Classified by role

Role	Systems (evidence)
Foundational	akira-store; <code>contracts/events.ts</code> (98 event constants); <code>shared/types/store-types.ts</code> (highest fan-in, 48 importers); SQLite connection
Domain-specific	akira-os modules; the vault; sessions with 30-day retention; conversations
Infrastructure	persistence, instrumentation, observability, the batch transaction (<code>genesis/batch.ts</code>)

Cognitive	GENESIS memory pipeline, understanding, identity, insights, context engines
Coordination	<code>routes/__root.tsx</code> (boot and lifecycle), <code>akira-store</code> , <code>genesis/composition.ts</code> , <code>genesis/batch.ts</code> , <code>companionStateService</code> (session lifecycle), <code>aiContextEngine</code> (per request)
Execution	AKIRA OS server functions and repositories; store actions; the provider <code>fetch</code>
Persists state	Only the repositories under <code>src/persistence/repositories</code> , plus <code>instrumentation/event-store</code> for the events table
External services	<code>genesis/context/ai/providers/*</code> (Gemini, OpenRouter) and <code>provider-manager.testConnection</code> . Nothing else calls out.

Complete system inventory

One row per significant system. GENESIS and AKIRA OS are broken down engine by engine in Parts 3 and 4. "Authority" lists what the code lets the system decide or change, not what it was meant to do.

System · location	Owns / purpose	Inputs → outputs	State	Communicates by	Authority	Not its job (by current boundary)
Client store persistence/akira-store.ts live	Workspace truth (projects, tasks, notes, sessions, chat mirror, profile, vault, memory stream)	Route actions → new state, persist() RPCs, platform events, saveMemory	Ephemeral in memory; durable through the repositories; one-way hydrated flag	subscribe / useSyncExternalStore , publish , dynamic import of akira-os services	Mutates all workspace state; decides what persists; applies GENESIS retention to the stream (applyDurableRetention)	Cognition, model calls
AKIRA OS modules src/akira-os/* live	Domain write paths and reads	Service call → createServerFn → repository	None (vault files on disk)	RPC; vault publishes on the server bus	Writes its own tables	Client state
Persistence src/persistence live	Schema, migrations, repositories, hydration, reset	Repository calls → SQL	SQLite (WAL, FK on)	Direct calls from server functions	Schema and data; startup purge; archive sync	Business rules (though retention and the archive mirror live here)
Instrumentation src/instrumentation live	Platform event bus, publisher, server event intake	publish(EventInput) → subscribers; RPC to the server bus	Subscriber sets; events table; timeline	Bus, RPC	None beyond delivery and storing events	Interpreting events

GENESIS memory pipeline <code>genesis/{events,candidate,validation,memory,retention,importance,recall,stories,batch,composition}</code> live	Turning events into memories, stories, importance and recall candidates	Events → derived structures → context package	All in memory; rebuilt from <code>genesis_memories</code>	Per-service <code>subscribe</code> , <code>runBatched</code> flushers	Promote/hold/reject; eviction; ranking	Writing workspace rows
Understanding · identity · insights <code>genesis/{understanding,identity,insights}</code> live	Understandings, emergent and foundation identity, insights	Memories and stories → understandings → prompt sections	In memory; foundation identity is never cleared	<code>subscribe</code> ; direct calls; <code>eventService.record</code> for identity events	Records <code>identity.*</code> events into the durable stream	—
GENESIS context engines <code>genesis/context/{state,habits,relationships,context-resolution,initiative} + insights/reflection</code>	Session and situational context	Store, presence → <code>ResolvedContext</code>	In memory, session-scoped	Per-context event objects	Companion state owns memory replay; initiative decides "Silence"	Acting on the user's behalf
Context assembly and AI engine <code>genesis/context/{context-builder,context-service,context-rules,context-relevance-selector,historical-search-intent,ai/*}</code> live	What the model sees, and the provider call	Prompt → system instruction → provider → response	Provider config in settings (keys included)	Pull per request; provider <code>fetch</code>	Decides history search, prompt content, active provider	Persisting replies (chat.tsx does that)
Contracts <code>src/contracts</code> , <code>shared/genesis-provider.ts</code> live	Seams between GENESIS and AKIRA OS	Registration → providers	Module-level singletons	Register/get	None	—

Observability src/observability inert	Health, telemetry, bus delivery	Bus deliveries and write outcomes → telemetry store	In memory	Delivery observer; registry listeners	None; nothing reads its output	—
Platform runtime src/runtime + compatibility, sdk, analytics unreachable	Module lifecycle, capabilities, SDK, metrics	—	—	Legacy string-topic bus (shared/infrastructure/event-bus)	Designed as a global module orchestrator	—
GENESIS orphans genesis/{planning, reasoning, decision} , context/{intelligence, relevance, assembly} , insights/reflection/engine unreachable	Plan, milestone, task and dependency; reasoning; decision strategies	—	—	—	Planning contains about 30 record() calls	—

GENESIS deep audit

GENESIS is 207 files, and 199 are reachable through the `genesis/index.ts` barrel. Reachable is not the same as effective, though: several engines below are wired in and produce nothing. Importing `@/genesis` composes the system (`genesis/index.ts:162` calls `composition.ts:initializeGenesisCognition`). That initializes 12 processors in order, each under its own try/catch, with the reality adapter attached last. The session and context engines are not part of that composition; `routes/__root.tsx` boots them separately.

LLM interaction: none in any engine. The only model calls are in `context/ai/providers` .

3.1 Memory pipeline engines

Engine · location	Purpose / processing	Inputs → outputs (consumers)	State / persistence	Trigger · lifecycle	Events	Failure
Reality adapter <code>events/reality-adapter.ts</code> GenesisRealityAdapter	Boundary from platform events into GENESIS. It drops duplicate ids (FIFO of 5,000), translates the event, then records it. The file's "DORMANT" header is stale: <code>composition.ts:104</code> attaches it.	Platform <code>AkiraEvent</code> → <code>eventService.record</code>	Seen set, counters; none durable	<code>attach()</code> from composition	Consumes <code>globalEventBus</code> (all events)	Swallows every error into counters and never logs. An event that fails downstream is still marked seen.
Event translation <code>events/event-translation.ts</code>	Pure TRANSLATORS table for 9 of the 98 event types: project created, updated, continued and deleted; note created, edited and deleted; task.completed; mission.completed.	type + payload → MemoryEvent fields	None	Per event	—	Throws; the adapter catches

eventService events/event-service.ts	The single recorder. It builds a MemoryEvent, classifies durability, persists it unless Transient, and fans out to callbacks, all inside runBatched .	~83 record() sites → saveMemory ; onRecord : candidateService, chatDeclarationPromoter, chat.tsx, brain.tsx	Callback set; durable through the store	Synchronous per call	Emits through onRecord	Persistence and each callback are guarded and logged
Chat declaration promoter events/chat-declaration-promoter.ts	Turns a chat turn into a declaration. Only the Goal category is promoted, as declaration_captured (Core).	chat_message → nested record	None	First processor in composition	Consumes onRecord	Via eventService guard
candidateService candidate/candidate-service.ts , candidate-rules.ts	First matching rule maps event type to a reason: Milestone, Goal Progress, Reflection Worthy or Repeated Activity. chat_message matches no rule.	MemoryEvent → MemoryCandidate (validationEngine, UI)	Up to 500 candidates in memory	Self-subscribes at module load	onRecord → subscribe	Listener guards
validator validation/validator.ts	Promote, Hold or Reject, from the first rule that isn't Hold. Two branches can never fire on platform events: the Goal-Progress title check and "0 minutes", because translated titles are fixed.	Candidate → outcome	None	Per candidate	—	No guard

memoryService memory/memory- service.ts	The root memory store and the reconstruction lifecycle. Promotes a candidate, then applies runtime retention (Core 2,000, Episodic 500).	Candidates → Memory (relationships, stories, importance, understanding, recall, UI); evictions → retentionService; clear → 5 subscribers	memories[] in memory; ids are regenerated on every replay	initialize() = clear, then replay of the reversed stream. Called from companionStateService.bootstrap (at boot and on every New Chat) and after hydration (__root.tsx:242).	subscribe, subscribeEviction, subscribeClear	Listener guards; an empty stream skips the clear
Memory relationships memory/relationships/*	Links between memories (Part Of, Continues, Caused By, References), at most 8 per memory	Memory → relationship (storyBuilder, UI)	Cache and index in memory	Module load and composition	subscribe	Guards
Stories stories/story-service.ts , story-builder.ts	Project arcs and one "Personal Growth Reflections" arc. At most 100 stories and 200 members each. No production code ever changes a story's status.	Memory and relationship → story (importance, identity, understanding, recall, context)	In memory	Composition; touchStory is deferred into the batch	subscribe, subscribeEviction	Guards; forgetMemories notifies no one
Importance importance/*	Signals per memory, not a single score: Milestone 0.9; Recency 1.0/0.8/0.6/0.3; Relationships; Story Influence; Reinforcement; User Intent 0.8	Memory and story → profile (recall, UI)	In memory; 20 history entries	Inline on promotion, plus a batch flush	subscribe	Guards

Recall recall/recall-builder.ts , recall-rules.ts , recall-service.ts	Candidates from an Active Story rule and a Multi-Factor composite, with weights per mode (BOOTSTRAP, QUERY, CONTINUATION). Ranked later in context-rules (12 slots, 3 reserved for authored memories).	Memories, stories, importance → recall candidates (contextBuilder, companion state, UI)	In memory	Not in composition: initialized and disposed by companion state	subscribe	Guards
Retention retention/policy.ts , retention-service.ts	Durability table; caps on the durable stream (Core 2,000, Episodic 750, Transient 0); fanning evictions out to the derived stores. An unknown event type defaults to Episodic .	Evictions → forgetMemories on three stores	Policy constants	Composition	subscribeEviction	Guard per store
Batch batch.ts	Synchronous transaction with phase order: stories → importance → understanding → identity → recall → context, up to 8 passes	markDirty → flushers	Depth counter and dirty set	Every record() and the replay	—	Guard per flusher; logs a flush that doesn't converge

3.2 Understanding, identity, insights

Engine · location	Purpose / processing	Inputs → outputs	State / persistence	Authority and side effects
understandingEngine understanding/engine.ts , rules.ts , builder.ts live	Builds the understanding graph from memories and stories with these rules: project, goal, knowledge, habit, relationship, preference and personalDeclarationRule . Computes Active and Archived status.	→ [UNDERSTANDINGS] prompt block; insightEngine	In memory, rebuilt on replay	personalDeclarationRule (rules.ts:618) writes the foundation identity store (createIdentity , createGoal , createInterest , ...), and each of those calls eventService.record(identity.*)

Emergent identity <code>understanding/identity-service.ts</code> , <code>identity-builder.ts</code> live	Observations from two story rules plus declarations	→ <code>contextBuilder</code> , <code>chat.tsx</code> , <code>brain.tsx</code> , <code>sessions.tsx</code>	In memory; cleared on memory clear	In memory only
Foundation identity <code>genesis/identity/*</code> : a façade over 14 services and <code>InMemoryIdentityRepository</code> live, partly	Identity graph, evidence, confidence, evolution, goals, interests, preferences, values, habits. Skill, relationship, personality, update and archive paths are not reached.	Read by <code>context-rules.ts</code> (goals, nodes, confidence)	Process memory only, never cleared , so it can outlive evicted declarations until restart	Records <code>identity.*</code> events (Episodic by default, so persisted) that nothing replays back into it
Intent resolver / classifier <code>understanding/intent-resolver.ts</code> , <code>intent-classifier.ts</code> live	Keyword heuristics. An ambiguity dictionary (pilot, python, apple) triggers a clarification block.	→ selector, prompt builder	None	Can make the model ask for clarification
hypothesesService <code>understanding/hypotheses.ts</code> inert	Onboarding hypotheses	3 readers; <code>proposeHypothesis</code> has no caller	Always empty	—
insightEngine <code>insights/insight-engine.ts</code> live	Rules over understandings: parallel commitments, learning momentum, goal alignment	→ <code>[INSIGHTS]</code> , not gated by relevance	In memory	Read-only
reflectionService <code>insights/reflection/service.ts</code> inert	Historical reports	No producer (<code>finalizeSession</code> , <code>setHistoricalReports</code> uncalled)	Always empty	Its Core <code>note_created</code> write path is uncalled

3.3 Session and context engines

Engine · location	Purpose / processing	Inputs → outputs	State	Lifecycle and authority
-------------------	----------------------	------------------	-------	-------------------------

companionStateService context/state/service.ts live	Focus, active project and active goal. Driven by store transitions in both directions.	Store, presence, recall → state (contextResolution, recall-rules through shared/genesis-provider)	In memory, rebuilt per bootstrap	Owns the memory layer's lifecycle. bootstrap() initializes validation, recall, memoryService (a full replay) and identity foundation. closeSession() disposes validation and recall. Called at boot and on every New Chat (chat.tsx:591-592). Records the Transient companion_bootstrapped .
relationshipService context/relationships/service.ts live	Contacts from @Name mentions in store.chat , with no role filter , so an @mention in an AKIRA reply also counts	→ contextResolution (Contact lines)	Rebuilt from the open conversation's chat	Session lifetime
habitService context/habits/service.ts live	"Workspace Focus Switch" on deliberate project switches. Boot, hydration and delete reassignment are excluded.	→ contextResolution	Session-scoped by design (documented)	runDecayCheck is never called, so habits never decay
contextResolutionService context/context-resolution/* live	Merges presence, companion state, relationships, habits and reflection	5 event sources → ResolvedContext (engine, initiative)	Cached, rebuilt on events	Read-only aggregator
initiativeService context/initiative/* inert	Proactivity decision. Its outcome is effectively always "Silence": the only branch that would ask a question needs conflictsExposed , which is never filled.	→ initiativeEvents , which has no subscriber ; getDecision has no caller	In memory	The one designed decision-maker for proactive action, and it has no effect
contextBuilder / contextService context/context-builder.ts , context-service.ts , context-rules.ts live	Builds the ContextPackage (recalled memories, goals, preferences, constraints, identity traits, arcs)	Recall, stories, emergent identity → package (chat.tsx, brain.tsx)	One active package	Batch phase "context"

Historical recall (GENESIS side)

context/historical-search-
intent.ts , ai/context-
engine.ts:recallHistory

live

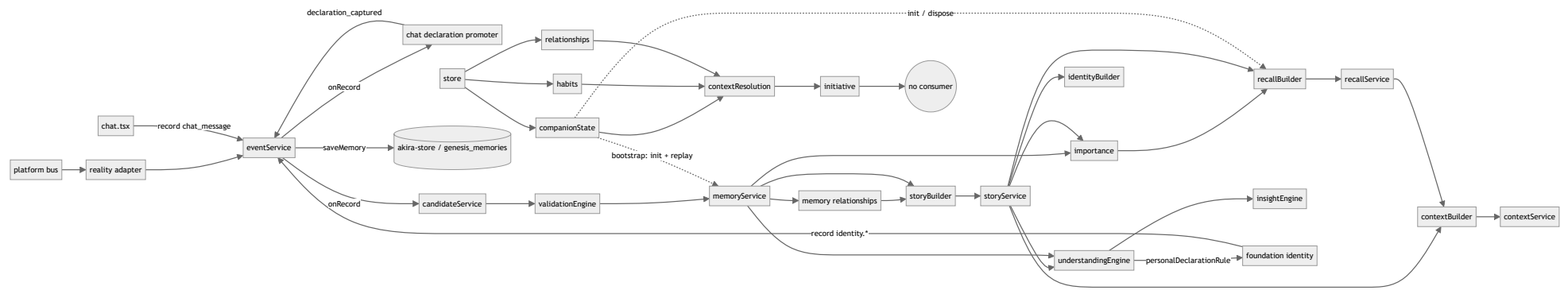
Explicit search, or automatic search
when the question is about the
user's past and genesisCovers()
is false

→ [FROM PAST
CONVERSATIONS]

None

Per request; never writes

3.4 How the engines interact



AKIRA OS deep audit

Every module follows the same three-part pattern: a client `services/` wrapper, a `server/` `createServerFn` , and a repository. Writes start as store actions that call `persist(op, () => import("../akira-os/<mod>").then(svc => ...))` . Reads go from the route to the service directly.

Component	Purpose	Server functions	State / persistence	Events	Consumers
projects	Project write-through	add, update, delete (transaction: clears <code>active_session</code> , stamps <code>sessions.project_deleted_at</code>), touch	<code>projects</code>	Store publishes <code>PROJECT_*</code>	akira-store
tasks	Task write-through and ordering	add, update, delete, reorder	<code>tasks</code> including <code>position</code>	Store publishes <code>TASK_*</code>	akira-store
notes	Note write-through	add, update, delete	<code>notes</code>	<code>NOTE_*</code>	akira-store
sessions	Work sessions, 30-day retention after a project is deleted	start, end, updateActiveTask	<code>sessions</code> , <code>settings.active_session</code> <code>n</code>	<code>SESSION_*</code>	akira-store; <code>session-outcomes.ts</code> for <code>sessions.tsx</code>
settings	Key-value blobs	profile, last project, chat, streaks, memories, generic get/set/delete (which refuse the archive key)	<code>settings</code>	<code>SETTINGS_UPDATED</code>	akira-store; provider-manager; <code>chat.tsx</code>
conversations	Chat archive: blob plus table mirror; Historical Recall provider	<code>persistSaveConversations</code> , <code>querySearchMessages</code>	<code>akira:chat:history:v1</code> , <code>conversations</code> , <code>chat_messages</code>	—	<code>chat.tsx</code> ; GENESIS through the contract
search	FTS search and history	run search, history get/delete/clear	<code>fts_workspace</code> , <code>search_history</code>	<code>SEARCH_EXECUTED</code> (server branch only)	<code>search.tsx</code> , <code>CommandPalette</code>
timeline	Timeline read model	<code>getTimelineEvents</code>	<code>timeline_events</code>	Subscribes on the server bus	<code>timeline.tsx</code>

vault	Files, folders, tags	14 RPCs (zod-validated upload and raw read)	<code>vault_*</code> tables; files under <code>AKIRA_VAULT_PATH</code> or <code><db dir>/Vault</code>	VAULT_* on the server bus; DB triggers write the timeline	useVault, useUploadQueue, VaultWorkspace
presence	Presence context (session type, return state, time period)	none	none	PRESENCE_UPDATED (transient, client only), every 30 s and on store change	companion state, contextResolution
tools	Static tools catalogue	none	none	—	tools.index.tsx. <code>enabled</code> is never consulted; <code>/tools/diagnostics</code> has no route.

Cross-cutting OS components

Event infrastructure

`instrumentation/event-bus.ts` has no topics: every subscriber gets every event. There are two instances, browser and server. `publisher.ts` runs middleware, then the local bus, then the `persistPublishEvent` RPC for non-transient events.

Runtime services

The real runtime is `__root.tsx`'s two effects (Part 5). The designed runtime in `src/runtime` (RuntimeManager, LifecycleManager, CapabilityRegistry) is `unreachable`.

Configuration

AI config in `settings akira:ai:*`; dev mode in `localStorage` plus the DOM event `akira:dev_mode_change`; `VITE_*` key fallback; nitro build preset `cloudflare-module` (`.output/nitro.json`).

Observability

A health registry written by the store and by the bus delivery observer. The telemetry store is write-only, and no UI reads either.

Scheduling

No scheduler. Timers: the presence 30 s decay interval, UI debounces (150, 250, 800 ms), the 500 ms "preparing" delay in chat, and the mock reply stream. The interval in `AnalyticsEngine` is unreachable.

External integrations

Only the two LLM providers. Vault thumbnails use a Web Worker (`workers/thumbnail-worker.ts`).

PART 5

User message runtime flow

The entry point is `routes/chat.tsx → handleSend` , the only production caller of `aiContextEngine.executeRequestStream` . `executeRequest` has no caller.

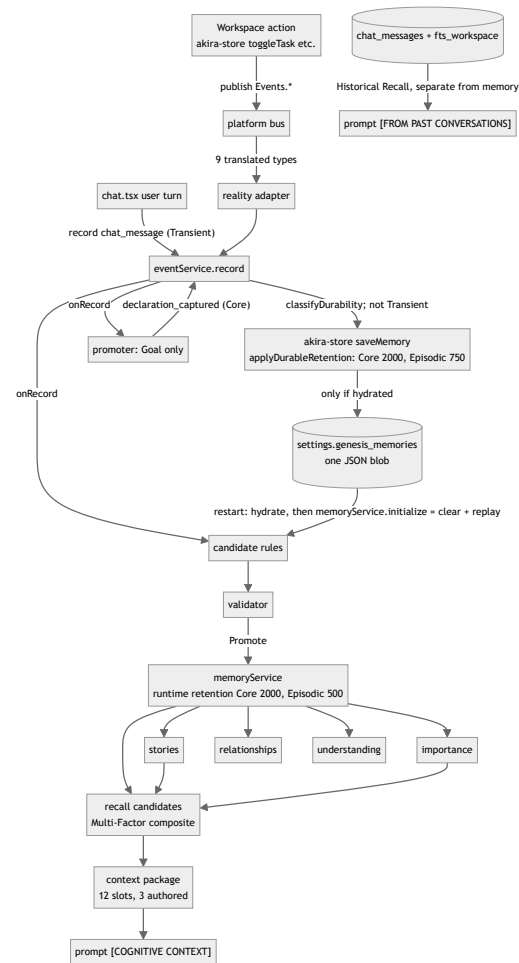
User types and presses Enter

```
|
▼ routes/chat.tsx · handleSend
|   resolve or create conversation id (convId); a new one gets generateTitle (string truncation, no model)
|   saveConversations(...)      → conversationsService.save → RPC persistSaveConversations
|                               → settings blob + conversations/chat_messages rows + fts_workspace (one transaction)
|   akira.clearChat() if new → store chat = ["Conversation cleared"] → persist settings.updateChat
|   akira.addChatMessage(user) → store chat += msg → persist settings.updateChat (whole array)
|   project name in text?      → akira.startSession(...) → sessions.start + SESSION_STARTED (chat.tsx:664-674)
|   eventService.record("chat_message", ...) (bypasses the platform bus)
|   |─ Transient: not persisted
|   |─ candidateService: no matching rule
|   |─ chatDeclarationPromoter: Goal declaration? → record("declaration_captured") Core → full pipeline (Part 6)
|
▼ 500 ms "preparing" delay; akira.addChatMessage("akira", "") placeholder
|   currentContext = contextService.getActiveContext() (package built at the last rebuild, not now)
|
▼ genesis/context/ai/context-engine.ts · executeRequestStream
|   1 intentResolver.resolveIntent(prompt, history)      keyword heuristics, ambiguity dictionary
|   2 contextResolutionService.getContext()              cached ResolvedContext
|   3 contextRelevanceSelector.selectContext(...)        strips project material when not workspace-relevant
|   4 recallHistory(prompt, conversationId, selection)
|       explicit search OR (past-question AND NOT genesisCovers)
|       → HistoricalRecallProvider → RPC querySearchMessages → FTS5/bm25, 5 results, current conversation excluded
|   5 promptBuilder.buildSystemInstruction(...)          [USER] [INTENT] [COGNITIVE CONTEXT] [UNDERSTANDINGS]
|                                                       [INSIGHTS] [RESOLVED CONTEXT] [FROM PAST CONVERSATIONS] [AVAILABLE PROJECTS]
|   6 providerRegistry.getActiveProvider().generateContentStream(request, onChunk, signal)
|
▼ providers/gemini-provider.ts | openrouter-provider.ts
|   browser fetch → generativelanguage.googleapis.com (key in URL) | openrouter.ai (Bearer header)
|   no key → local "[MOCK RESPONSE]" streamed word by word
|
▼ onChunk (chat.tsx)
|   akira.updateChatMessage(id, text, skipPersist=true); saveConversations(..., persist=false)
|
▼ finally
|   akira.updateChatMessage(id, text) → persist settings.updateChat
|   saveConversations(..., true)      → archive blob + mirror + fts
|   failure → describeProviderFailure → "🚨 System Note" stored as the AKIRA message
```

|
▼ Response shown. The AI reply is NOT recorded as a GENESIS event or memory.

The recall a chat turn sees can be stale. A chat turn without a Goal declaration changes no memory, so no recall rebuild runs. The prompt uses the recall candidates and context package scored at the last rebuild (traced in the memory-pipeline audit). Historical Recall is the only retrieval that runs fresh on each turn.

Memory flow



Question

Answer from code

Where is a memory created?	<code>memoryService.processCandidate</code> , after <code>validator.validate</code> promotes it
What decides whether something becomes memory?	Durability class (<code>retention/policy.ts:classifyDurability</code>); a matching candidate rule (<code>candidate-rules.ts</code>); the validator outcome. Chat becomes memory only as a Goal declaration.
Where is importance decided?	<code>importance-rules.ts</code> signals, evaluated in <code>importanceBuilder.evaluateMemoryImportance</code> . There is no single score.
Where is it persisted?	Only as the event stream: <code>akira-store</code> <code>saveMemory</code> → <code>settings.genesis_memories</code> . Memories, stories, importance and recall are never stored.
How is it retrieved?	<code>recallBuilder.rebuildRecallCandidates</code> → <code>recallService</code> → <code>context-rules.filterActiveRecallCandidates</code>
How does it enter context?	<code>contextBuilder.rebuildContextPackage</code> → <code>contextService</code> → chat.tsx passes it in → <code>promptBuilder.serializeContextPackage</code>
How does historical search work?	A separate path. The <code>chat_messages</code> FTS index is reached through <code>HistoricalRecallProvider</code> . The evidence is quoted into one request and never becomes memory.
Across sessions	"Session" here means a New Chat: <code>closeSession</code> then <code>bootstrap</code> , which replays the whole stream again.
After restart	Hydration loads the stream, then <code>memoryService.initialize</code> rebuilds every derived structure. Memory, story and relationship ids are regenerated. Transient events and anything beyond the caps are gone.
Who owns memory?	<code>memoryService</code> owns runtime memory; <code>akira-store</code> owns the durable stream; <code>retention/policy.ts</code> owns the rules for both. That is three places.
Who only consumes?	Stories, importance, recall, understanding, insights, context, UI (brain.tsx, chat.tsx, sessions.tsx)

Observed (traced statically, not run). The foundation identity repository is never cleared, and every replay re-runs `personalDeclarationRule` . On each cold start or New Chat, a stored declaration can therefore create its identity, goal and interest again, recording fresh `identity.*` events. Those default to Episodic, so they're persisted, and they use up the 750 Episodic slots without producing any memory. Whether this actually accumulates needs a runtime check (Part 20).

Context flow

Context source	Built by	Refreshed when	Prompt section	Filtered by relevance?
Saved user name	store <code>profile</code> , if <code>profileSaved</code>	Per request	<code>[USER]</code>	No
Intent	<code>intentResolver</code>	Per request	<code>[INTENT RESOLUTION METADATA]</code> or the clarification block	—
Recalled memories, goals, preferences, constraints, identity traits, arcs	<code>contextBuilder</code> from recall, stories, identity (emergent and foundation)	On the last rebuild (event driven)	<code>[COGNITIVE CONTEXT]</code>	Stories and project goals stripped when not workspace-relevant
Understandings	<code>understandingEngine</code>	On memory or story change	<code>[UNDERSTANDINGS]</code>	Project category removed
Insights	<code>insightEngine</code>	On understanding change	<code>[INSIGHTS]</code>	No
Presence, companion state, relationships, habits, reflection	<code>contextResolutionService</code>	On their events (cached)	<code>[RESOLVED CONTEXT]</code>	Active project, focus and habits stripped
Past conversations	Historical Recall provider	Per request, when triggered	<code>[FROM PAST CONVERSATIONS]</code>	Only runs on an explicit request or the automatic rule
Projects list	store <code>projects</code>	Per request	<code>[AVAILABLE PROJECTS]</code>	Only if workspace-relevant
Conversation history	<code>chat.tsx</code> <code>currentHistory</code>	Per request	Provider <code>history</code> (not in the system instruction)	—
System persona	A hardcoded string in <code>chat.tsx</code> (l.776)	—	Base instruction	—

Model invocation: `providerRegistry.getActiveProvider()` . `contextPackage` is also passed to the provider as an object, but providers read it only to build the mock reply.

State flow

State	Created	Stored	Owner	Readers	Writers	Lifetime / recovery
Workspace (projects, tasks, notes)	Store actions	SQLite rows	akira-store	Routes, GENESIS via WorkspaceProvider	Store → repositories	Durable; hydrated at start
Work session	<code>startSession</code> (also auto on project page mount and on a chat project match)	<code>settings.active_session</code> , <code>sessions</code>	akira-store / sessions repository	Routes, presence, companion state	Store; project delete	Durable; finished sessions retained 30 days after project delete; purge at boot. Store and DB ids differ until reload.
Companion state	<code>bootstrap()</code>	Memory	companionStateService	contextResolution, recall-rules	Store transitions	Per process / New Chat; rebuilt
Memories and derived cognition	Pipeline	Memory; stream in <code>genesis_memories</code>	memoryService (runtime), store (stream)	Context, UI	eventService	Rebuilt by replay; bounded
Foundation identity	personalDeclarationRule	Memory (<code>InMemoryIdentityRepository</code>)	identity façade	context-rules	understanding rule	Process lifetime; never cleared
Goals	Project arcs (stories) plus foundation <code>IdentityGoal</code> plus Goal understandings	Memory	Split across three systems	context-rules, prompt	Pipeline	Rebuilt
Contacts	@mentions in store chat	Memory	relationshipService	contextResolution	Store chat	Rebuilt from the open conversation only

Habits	Project switches	Memory	habitService	contextResolution	Store transitions	Session-scoped
Open conversation	chat.tsx	<code>settings.chat</code>	akira-store	Relationships, presence, recall	chat.tsx	Durable; replaced on conversation switch
Chat archive	chat.tsx	Settings blob plus <code>chat_messages</code>	conversations module	chat.tsx, Historical Recall	<code>conversationsService.save</code> only	Durable; survives reset (open question)
AI configuration and keys	Settings UI	<code>settings</code> <code>akira:ai:*</code> , plaintext	aiProviderManager	Providers in the browser	Settings route	Durable
Pending operations	<code>persist()</code>	<code>writeChain</code> and <code>pendingPersistence</code> in memory	akira-store	Tests only	Store	Lost on tab close; no outbox, no retry
Runtime health	Store, bus observer	Health registry (memory)	observability	Telemetry store only	observeWrite, EventBusObserver	Process; never shown
Missions	No mission object. "Missions" in the UI is tasks (<code>MISSION_COMPLETED</code> fires when all of a project's tasks are done). Plans and milestones exist only in unreachable <code>genesis/planning</code> .					

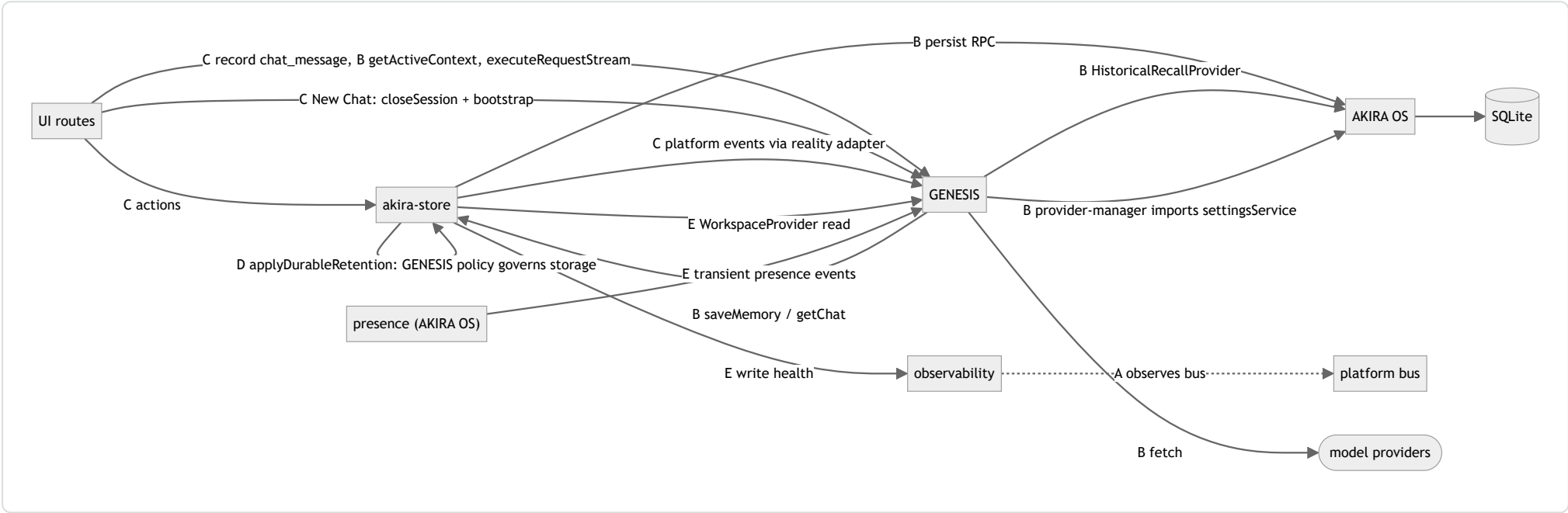
Event and communication audit

Every mechanism

#	Mechanism	Defined	Shape	Status
M1	Platform bus <code>globalEventBus</code>	<code>instrumentation/event-bus.ts</code>	No topics; delivers to all subscribers; one delivery-observer slot; browser and server instances	live
M1a	Publisher + RPC bridge	<code>instrumentation/publisher.ts</code> , <code>server/index.ts</code>	Middleware → local bus → <code>persistPublishEvent</code> (non-transient)	live
M2	Legacy string-topic bus	<code>shared/infrastructure/event-bus</code>	<code>subscribe(type)</code> , * wildcard	unreachable (runtime only)
M3	GENESIS <code>eventService</code>	<code>genesis/events/event-service.ts</code>	record + <code>onRecord</code> , persists and batches	live
M4	Presence channel	<code>contracts/presence-channel.ts</code>	Filtered view of M1	live
M5	Per-context event objects	<code>stateEvents</code> , <code>habitEvents</code> , <code>relationshipEvents</code> , <code>contextResolutionEvents</code> , <code>initiativeEvents</code> , <code>reflectionEvents</code>	Module-level Set	live ; <code>initiativeEvents</code> has 0 subscribers
M6	Service listener sets	<code>candidate</code> , <code>memory (+eviction, clear)</code> , <code>relationship</code> , <code>story</code> , <code>importance</code> , <code>recall</code> , <code>context</code> , <code>understanding</code> , <code>identity</code> , <code>insight</code>	<code>subscribe</code> → <code>unsub</code>	live
M7	AIPProviderManager listeners	<code>provider-manager.ts</code>	React hook	live
M8	Store listeners	<code>akira-store.ts</code>	<code>subscribe</code> , <code>useSyncExternalStore</code>	live
M9	Health registry listeners	<code>observability/health</code>	<code>subscribe</code>	live

M10	Two byte-identical loggers	<code>lib/logger.ts</code> , <code>shared/infrastructure/logger</code>	Listener sets	<code>inert</code> : no production writer
M11	DOM event <code>akira:dev_mode_change</code>	<code>settings.tsx</code> → Sidebar, tools.index	window event	<code>live</code>
M12	Server functions (RPC)	every <code>akira-os/*/server</code>	request/response	<code>live</code>
M13	Provider registries	WorkspaceProvider, HistoricalRecallProvider, <code>shared/genesis-provider</code> , providerRegistry	register/get singletons	<code>live</code>

Communication map across systems



Letters: **A** internal · **B** dependency · **C** coordination · **D** authority · **E** data sharing.

Interaction	Class	Direction	Evidence
Workspace mutation becomes cognition	C	AKIRA OS → GENESIS	akira-store <code>publish</code> → reality adapter → <code>eventService.record</code>
Chat turn becomes cognition	C	UI → GENESIS	<code>chat.tsx:695</code> records directly, bypassing the bus
Presence signal	E	AKIRA OS → GENESIS	<code>presence/service.ts:86</code> → presence channel
GENESIS reads workspace	E / B	GENESIS → store	WorkspaceProvider, 9 consumers
GENESIS writes the durable stream	B	GENESIS → store	<code>shared/genesis-provider.saveMemory</code>
Retention policy applied to storage	D	GENESIS policy → AKIRA persistence	<code>akira-store.ts:2</code> imports <code>applyDurableRetention</code>
Session reset	C / D	UI → GENESIS memory layer	<code>chat.tsx:591-592</code> → full replay
Historical recall	B	GENESIS → AKIRA OS	Contract registered at <code>akira-os/conversations/services:29</code>
AI config storage	B (bypasses the contracts)	GENESIS → AKIRA OS	<code>provider-manager.ts:76,220</code> dynamic <code>@/akira-os</code>
Type dependency on presence	B (types)	GENESIS → AKIRA OS	10 GENESIS files import <code>akira-os/presence/types</code>
Client events to server	E	Client → server bus	<code>persistPublishEvent</code>

Authority map

System	Local decisions	Cross-system decisions	Triggers work	Modifies global state	Starts work on its own	Creates goals	Calls external agents/tools
UI routes (chat.tsx, projects.\$id.tsx, __root)	Conversation, title, project match	Starts and ends sessions; resets the companion session; triggers replay	Yes	Yes, through the store	Mount effects only	No	Starts the LLM call
akira-store	Every workspace mutation; what persists	Applies GENESIS retention; nulls references on project delete	RPCs, events	Yes (owner)	No	No	No
AKIRA OS server functions	Row-level writes, vault file I/O	Project delete clears the running session and stamps retention	Server events (vault)	Database	Startup purge and sync (initializer)	No	No
GENESIS memory pipeline	Promote, hold or reject; eviction; ranking	Shapes what the model sees	Derived rebuilds	The durable stream (via the store)	No	Indirectly, as Goal understandings and arcs	No
Understanding / identity	Understanding status; identity creation	Writes <code>identity.*</code> into the durable stream	Records	The stream	No (replay re-runs it)	Creates <code>IdentityGoal</code>	No
companionStateService	Focus, active project	Initializes and disposes the memory and recall engines	Full replay	Session state	No	No	No
contextResolutionService	Narrows contacts and habits to the active project	None (read-only)	Initiative evaluation	No	No	No	No

initiativeService	Proactivity (always Silence)	None reaches anything	No	No	Re-evaluates on events, with no effect	No	No
aiContextEngine	History search; prompt content	What the model is told	Provider call	No	No	No	Yes: the model, through the provider
aiProviderManager	Active provider, keys	—	Test pings	Settings keys	No	No	Pings providers
Initializer	Schema and migrations	Purges expired sessions; syncs the archive; backfills FTS	At boot	Database	Yes, at boot	No	No
Model	Wording only	None. Output is never parsed for actions.	No	Its reply is saved as chat	No	No	No tool calling

Nothing in the system can start work on the user's behalf. The two components designed for that are `initiativeService` (inert) and the unreachable `genesis/planning` and `runtime`. Everything else either reacts to a user action, runs at boot, or runs on the 30 s presence timer.

LLM and model usage

Call site	Provider · host	Purpose	Input → output	Stream
<code>providers/gemini-provider.ts</code> <code>generateContent / generateContentStream</code>	Gemini · <code>generativelanguage.googleapis.com</code> , model hardcoded <code>gemini-2.5-flash</code> , key in the URL query	Chat	history + prompt + systemInstruction → normalized response	Yes (JSON brace parser)
<code>providers/openrouter-provider.ts</code>	OpenRouter · <code>openrouter.ai/api/v1/chat/completions</code> , Bearer header, model from config	Chat	system + history + user → normalized response	Yes (SSE)
<code>provider-manager.ts:testConnection</code> ×2	Both	Settings "Test connection"	"ping" → status	No

- **Abstraction:** the `AIPProvider` interface, `providerRegistry` (a Map with an active name), `aiProviderManager` (config, keys, status, metrics), `aiContextEngine` (single dispatch), and `response-normalizer` adapters.
- **Selection:** the provider is chosen centrally. The model and keys are not: the Gemini model is hardcoded and ignores config, default model names appear in three places, and provider names are string literals in the manager, `chat.tsx` and `settings.tsx`.
- **Keys:** stored in plaintext in `settings akira:ai:keys` , pulled into the browser by a GET RPC, and held in React state. All model calls are browser `fetch` ; there is no server proxy. A `VITE_*` fallback would put a key into the bundle at build time.
- **Errors:** mapped to user messages (no key, rate-limited, invalid key, network). No retry and no fallback between providers. Abort is supported.
- **Role:** a **pluggable text-generation service**. Nothing parses its output, and there is no tool calling (`finishReason: "tool_calls"` is typed but never produced or read).
- **Reply re-ingestion:** replies go to `settings.chat` and the archive, not to GENESIS. They come back three ways: as provider `history` ; as `AKIRA:` evidence in Historical Recall (no role filter); and as @mention contacts (no role filter).
- **Switching providers:** between Gemini and OpenRouter is a config change. A new cloud provider needs a class plus edits in the manager, `chat.tsx` and `settings.tsx`. A local or server-side provider is an architectural change, because every call is a browser fetch using a key held in the browser.

Persistence audit

Database: one SQLite file through `better-sqlite3` (`persistence/connection.ts` : WAL, foreign keys on, busy_timeout 5000). By default it lives at `%APPDATA%\AKIRA\akira.db` . On a fresh database `initializer.ts` runs `schema.sql` , then idempotent dynamic migrations on every boot. Only `schema_version` 1 exists; there is no numbered migration runner, and `migrations/002_file_vault.sql` has no loader.

Object	Created	Stored	Read	Modified	Deleted / expired
Project	<code>addProject</code>	<code>projects</code>	<code>getInitialState</code>	update, touch; session end adds time	delete (transaction); reset
Task	<code>addTaskDetails</code>	<code>tasks</code> (+position)	hydration (ordered by position)	toggle, update, reorder	delete; reset
Note	<code>addNote</code>	<code>notes</code>	hydration	update (800 ms autosave)	delete; reset
Session	<code>sessions.end</code>	<code>sessions</code> ; running one in <code>active_session</code>	hydration (hides expired)	project delete stamps <code>project_deleted_at</code>	<code>purgeExpired</code> at boot (30 days); reset
Memory stream	<code>eventService</code>	<code>settings.genesis_memories</code>	hydration → replay	Whole blob rewritten per event	Retention caps; reset
Open conversation	chat actions	<code>settings.chat</code>	hydration	Whole array rewritten	<code>clearChat</code> ; reset
Chat archive	<code>chat.tsx</code>	Settings blob + <code>conversations</code> / <code>chat_messages</code> + FTS	<code>chat.tsx</code> (blob); recall (tables)	<code>conversationsService.save</code>	Sidebar delete (mirror + cascade); not by reset
Profile	Settings save	<code>settings.profile</code>	hydration (+ <code>profileSaved</code>)	Settings	Never
Platform events	Server bus	<code>events</code> table (created lazily)	Unreachable analytics only	—	UNKNOWN

Timeline	TimelineSubscriber; vault triggers	<code>timeline_events</code>	timeline.tsx	—	Not by reset; delete callers UNKNOWN
Vault file	upload RPC	<code>vault_files</code> + disk	hydration, raw read	rename, move, tag	Soft delete, purge; not by reset
Search history	Server branch only	<code>search_history</code>	search.tsx	—	Cap 50. Never written from the UI: <code>persistRunSearch</code> calls the repository directly.
AI config	Settings	<code>settings akira:ai:*</code>	provider-manager	Settings	None
Browser storage	various	localStorage: dev mode, legacy state, migrations, thumbnails; sessionStorage: <code>akira:booted</code>	UI	—	—

Serialization: most cognition and chat state is a JSON blob rewritten in full on each change: memories, chat, the archive. Hydration validates that the array-shaped keys really are arrays (`parsePersistedArray`).

Failure and recovery

Mechanism	Where	What it actually does
Write failure	<code>akira-store.ts persist</code>	Logs, then <code>observeWrite</code> records a failure on <code>akira-store.persist.<op></code> (critical after 3). The chain always resolves, so later writes continue. In-memory state is not rolled back , so the store and the database can diverge until reload.
Retries	<code>SQLiteSearchRepository.executeWithRetry</code>	3 attempts on busy or locked, synchronous backoff. Search only. Nothing else retries, including model calls.
Pre-hydration guard	<code>saveMemory</code>	Refuses to write the stream before hydration, so an empty array can't overwrite history
Hydration failure	<code>__root.tsx</code>	Toast; <code>hydrated</code> stays false, so the UI stays blank and nothing is overwritten. No retry.
Unreadable archive	<code>syncFromArchive</code>	Logs and leaves the tables alone
Migration	<code>initializer.ts</code>	No per-step handling. A throw aborts the remaining steps and <code>server.ts</code> only logs it. The sessions rebuild restores its pragmas in <code>finally</code> and checks foreign keys.
Legacy migration	<code>migration-impl.ts</code>	Idempotent via <code>migration_history</code> , transactional, records failure
Timeline write failure	timeline repository	In-memory fallback queue, drained on the next insert. The <code>flush()</code> on shutdown never runs.
Event handling	reality adapter; bus	The adapter swallows errors silently and marks the event seen (no retry). The bus guards each subscriber.
Cognition	listeners, batch	Each listener and flusher is guarded; a flush that doesn't converge is logged
Model failure	chat.tsx	Classified message stored as a "System Note" reply; no fallback
Vault upload	VaultStorageService	Removes the copied file if the commit fails

Startup recovery	initializer, replay	FTS backfill, archive re-sync, session purge, full cognition replay
Shutdown	<code>__root.tsx</code> effect cleanup	Shuts down the 8 services. No server process hooks; <code>settlePendingPersistence</code> and <code>closeDatabaseConnection</code> are never called in production, so in-flight writes on tab close are not awaited.

Existing orchestration

Component	Location	Controls	Decides	Coordinates	Scope	Status
Root lifecycle	<code>routes/__root.tsx</code> effects	Legacy migration, hydration, cognition replay, 8 engine init/shutdown pairs, the hydration gate	Boot order	Persistence, GENESIS, presence, timeline	Global, client	live
Client store	<code>persistence/akira-store.ts</code>	All workspace mutation and write-through; event emission	What changes and persists	AKIRA OS services, GENESIS (through events and providers)	Global	live
GENESIS composition	<code>genesis/composition.ts</code>	Which 12 processors are live, in what subscription order	Wiring only	Memory pipeline	Global, one-shot	live
Batch transaction	<code>genesis/batch.ts</code>	When derived work runs, in phase order	Ordering, coalescing	stories → context	Global, synchronous	live
Companion session	<code>context/state/service.ts</code> bootstrap/closeSession	Memory-layer lifecycle; focus state	Focus and active project	validation, recall, memory, identity	Global	live
Context resolution	<code>context/context-resolution</code>	Situational context	Narrowing, certainty	5 context sources	Global, read-only	live
AI context engine	<code>context/ai/context-engine.ts</code>	The per-request pipeline	History search, prompt content	intent, selector, recall, prompt, provider	Request-local	live
Initiative	<code>context/initiative</code>	Nothing downstream	"Silence"	—	Global	inert
Event intake router	reality adapter; <code>persistPublishEvent</code>	Which platform events become cognition; server event fan-in	Translation, dedup	Bus to GENESIS; client to server	Global	live

Initializer	<code>persistence/initializer.ts</code>	Schema, migrations, purge, sync, backfill	Startup data repairs	Database	Global, server boot	live
Planner	<code>genesis/planning</code> (38 files)	Plan → Milestone → Task → Dependency → Blocker	Planning	—	—	unreachable
Module runtime	<code>runtime/</code> : RuntimeManager, LifecycleManager, CapabilityRegistry, DependencyResolver	Module discovery, ordering, lifecycle state machine, capabilities	Lifecycle transitions	Modules (none registered)	Global (designed)	unreachable
Decision / reasoning	<code>genesis/decision</code> , <code>reasoning</code>	Strategy registries	—	—	—	unreachable

There is no command router, mission manager or workflow engine in the live code. Within the reachable code, a "task manager" exists only as the tasks module (CRUD).

Candidates for the VAJRA role

This section presents evidence and tradeoffs only. It doesn't recommend a candidate.

aiContextEngine

Matches: the one place every model request passes; already decides what the model sees and when to search history.

Doesn't match: request-local and stateless; has no lifecycle or authority over other systems; chat-only.

Would move in: intent routing, any action dispatch. **Coupling risk:** it sits inside GENESIS, so control would be embedded in the cognitive layer.

routes/__root.tsx lifecycle

Matches: the de facto global lifecycle owner; boot order and engine start/stop.

Doesn't match: a React route effect; client-only; no decision-making; the route contract says routes hold no logic.

Coupling risk: high. Growing it means growing UI-owned control.

companionStateService

Matches: owns session lifecycle, including memory replay; holds focus and active-project state.

Doesn't match: it's a context engine; its control over upstream engines is an inversion of the pipeline.

Coupling risk: already the most surprising ownership edge (a downstream engine initializes upstream ones).

initiativeService

Matches: the only component designed to decide whether AKIRA should act on its own; already consumes the resolved situation.

Doesn't match: inert. Always Silence, no consumer, no action channel.

Would need: real decision inputs and an executor. **Coupling risk:** low; nothing depends on it.

contextResolutionService

Matches: a global aggregator of situational state from five sources.

Doesn't match: read-only by design; no authority.

Coupling risk: giving an aggregator authority mixes observing with commanding.

akira-store

Matches: the global state authority and the source of all workspace events.

Doesn't match: a state container, not a decision-maker; already 1.4k lines and high fan-out.

Coupling risk: high. It would concentrate state and control in one module.

src/runtime (RuntimeManager + LifecycleManager + CapabilityRegistry)

Matches: designed as exactly a central control layer (lifecycle, capabilities, dependencies) under accepted ADR-016..020.

Doesn't match: no production importer; uses the unreachable legacy bus; no live module is registered with it.

Tradeoff: activating it means the live systems have to adopt its module model.

genesis/planning

Matches: the only implemented objective model (plans, milestones, dependencies, blockers).

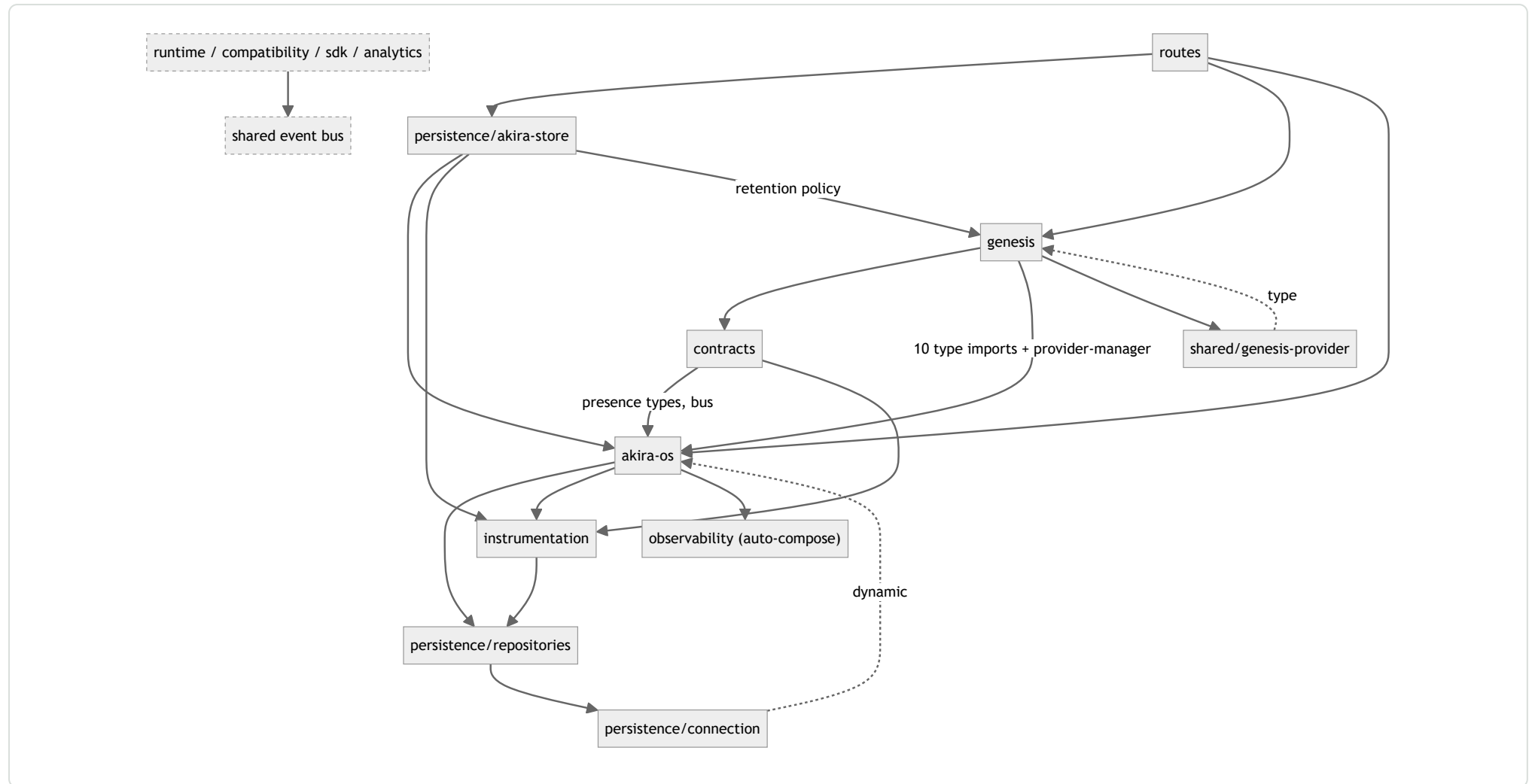
Doesn't match: unreachable; overlaps the identity goals and project arcs; its ~30

`record()` calls would write durable events.

Tradeoff: it's an objective model, not a controller.

Separate VAJRA versus evolving a candidate. The evidence points both ways. No live component holds cross-system decision authority today: control is spread across `__root.tsx`, the store, the companion session, the batch and the per-request engine, so a separate layer wouldn't duplicate any existing authority. But the `runtime/` stack and `initiativeService` are earlier attempts at parts of that role. Creating VAJRA alongside them without deciding their fate would leave three control designs in the tree.

Dependency graph



Highest fan-in (importers)

Highest fan-out (imports)

48 shared/types/store-types.ts	58 genesis/index.ts
43 lib/utils.ts	28 genesis/planning/index.ts
37 genesis/planning/types.ts (unreachable consumers)	24 observability/api/index.ts
34 contracts/events.ts	20 components/vault/VaultWorkspace.tsx
32 genesis/events/event-service.ts	19 genesis/identity/services/IdentityService.ts
26 persistence/connection.ts	16 genesis/context/state/service.ts
20 akira-os/index.ts	15 akira-store.ts , context-resolution/service.ts

Circular dependencies

- **Static, live:** routes/tools.vault.tsx → VaultWorkspace → FileGrid → Thumbnail → routes/tools.vault.tsx (7-file strongly connected component; a component imports from a route).
- **Through dynamic imports:** persistence/connection.ts → akira-os/search → persistence/repositories → Sqlite*Repository → connection.ts (20 files).
- **Runtime cycles through provider indirection:** recall-rules reads companion state through shared/genesis-provider , while companion state initializes recall and memory.
- **Unreachable:** runtime/module-context ↔ module-instance ; observability/logging/logger ↔ logger-registry .
- No cycle between the genesis, akira-os and persistence barrels. The two-way GENESIS ↔ AKIRA OS dependency runs only through type-only or leaf modules.

Central components: akira-store, eventService, memoryService, storyService. **Isolated components:** runtime, compatibility, sdk, analytics, planning, reasoning, decision (0 production importers).

Data ownership map

Data	Owner	Readers	Writers	Persistence	Lifetime
Memory (runtime)	memoryService	Pipeline, context, UI	validationEngine	Rebuilt	Process / New Chat
Memory stream	akira-store (rules: retention/policy)	Hydration, replay	eventService	genesis_memories	Durable, bounded
Context package	contextService	chat.tsx, brain.tsx, engine	contextBuilder	None	Until next rebuild
Resolved context	contextResolutionService	Engine, initiative	Itself	None	Cached
Goal	Split: stories (project arcs), foundation identity (IdentityGoal), understanding (Goal)	context-rules, prompt	Pipeline, personalDeclarationRule	Derived from the stream	Rebuilt
Relationship (contact)	relationshipService	Resolution	Store chat scan	Derived from settings.chat	Open conversation
Relationship (memory link)	memory relationships service	Stories, importance	Itself	None	Rebuilt
Companion state	companionStateService	Resolution, recall	Store transitions	None	Process / New Chat
Session (work)	sessions module + akira-store	Routes, presence	Store, project delete	sessions , active_session	30 days after project delete
Session (chat)	chat.tsx (conversation id)	Engine (exclusion)	chat.tsx	Archive	Until deleted
Knowledge	understandingEngine (Knowledge category)	Prompt	Rules	Derived	Rebuilt
Reflection	reflectionService	Resolution	No production writer	None	Always empty

Event (platform)	instrumentation	Reality adapter, timeline, events table	store, vault, presence, search	events , timeline_events	Retention UNKNOWN
Configuration	aiProviderManager; localStorage (dev mode)	Providers, UI	Settings route	settings akira:ai:*	Durable
Mission	No owner: no mission object exists. Tasks stand in for it.				
System state	Health registry (observability)	Telemetry only	Store, bus observer	None	Process

Actual architectural boundaries

System	Inside	Allowed dependencies (contract)	Observed dependencies	Public interface	Where observed ≠ intended
Routes	Mount containers	Store, services (MODULE_CONTRACT 2.2)	Store, services, 45 GENESIS imports in 5 files, <code>eventService.record</code>	Pages	chat.tsx (migration, project matching, auto-session, session reset); projects.\$id.tsx (auto session on mount); index.tsx (stats); a component imports a route (vault cycle)
akira-store	Workspace state	akira-os services	akira-os (38 dynamic imports), instrumentation, GENESIS retention policy	<code>akira</code> , hooks, providers	Persistence depends on a GENESIS policy module
GENESIS	Cognition	Contracts only; no DB writes (ARCHITECTURE §5)	0 imports of persistence, routes or sqlite; 12 imports of akira-os (presence types, provider-manager → settings)	<code>genesis/index.ts</code> barrel (58 imports)	provider-manager reaches AKIRA OS without a contract; GENESIS writes the durable stream through <code>genesis-provider</code>
Contracts	Interfaces and seams	Nothing	<code>presence-channel</code> imports the bus and akira-os types; <code>TimelineRepository</code> imports akira-os types	Providers, events	Contracts depend on an implementation
AKIRA OS	Domain services	Repositories	Repositories (42 dynamic imports), observability side effect	Services, server functions	—
shared/genesis-provider	Store ↔ GENESIS seam	(not in contracts/)	Type-imports GENESIS state	save/get memories, chat, companion state	A second, unofficial seam outside contracts

The creator's mental model

What AKIRA is. A local, single-user companion. The browser holds the whole working mind and the database holds its memory. React pages edit a workspace (projects, tasks, notes, sessions, files) in a client store that writes through to SQLite. Every meaningful workspace change is announced on an event bus. GENESIS listens, turns a narrow set of those announcements into memories, organizes them into stories and understandings, and keeps a ranked package ready for the next conversation.

How a conversation works. The chat page records the message and saves it to the archive. It then asks the AI engine to assemble a system instruction from what GENESIS already holds: the saved name, intent, recalled memories, goals, understandings, insights, situational context, and past-conversation evidence when the question calls for it. The instruction is streamed to Gemini or OpenRouter straight from the browser, and the reply is shown and saved as chat. The model decides nothing beyond its words.

How memory works. Only events are stored, as one bounded JSON stream. Everything AKIRA "knows" is recomputed from that stream at every start and on every New Chat. A chat turn becomes memory only when it states a goal. Past conversations are a separate, searchable archive that is quoted when asked about and never learned from.

How state and communication work. State lives in four places: the client store (workspace), the database (rows and blobs), GENESIS's in-memory structures (rebuilt), and session engines (reset per conversation). Systems talk through the store's subscriptions, the platform bus, the GENESIS recorder and per-service listener sets, all synchronously. Server functions carry everything to the database.

Where authority and orchestration live. They are spread out. The root page owns boot and engine lifecycle. The store owns every change to the workspace. The companion session owns the memory layer's lifecycle. A batch transaction orders cognitive rebuilds. The AI engine owns each request. Nothing coordinates across all of them, and nothing acts on its own.

When something fails. Writes fail quietly into a health registry nobody reads, and the screen keeps showing the change. Model failures appear as a system note. A broken hydration leaves the app blank rather than overwriting data. Nothing retries except search.

What it can't do today. It can't act or plan on its own: the initiative engine is inert and the planner is unreachable. It doesn't use tools. It doesn't learn from conversation beyond stated goals, and it doesn't keep observed habits. It can't run its model calls off the client. It can't surface its own health. It can't coordinate work across systems.

Unknown / requires verification

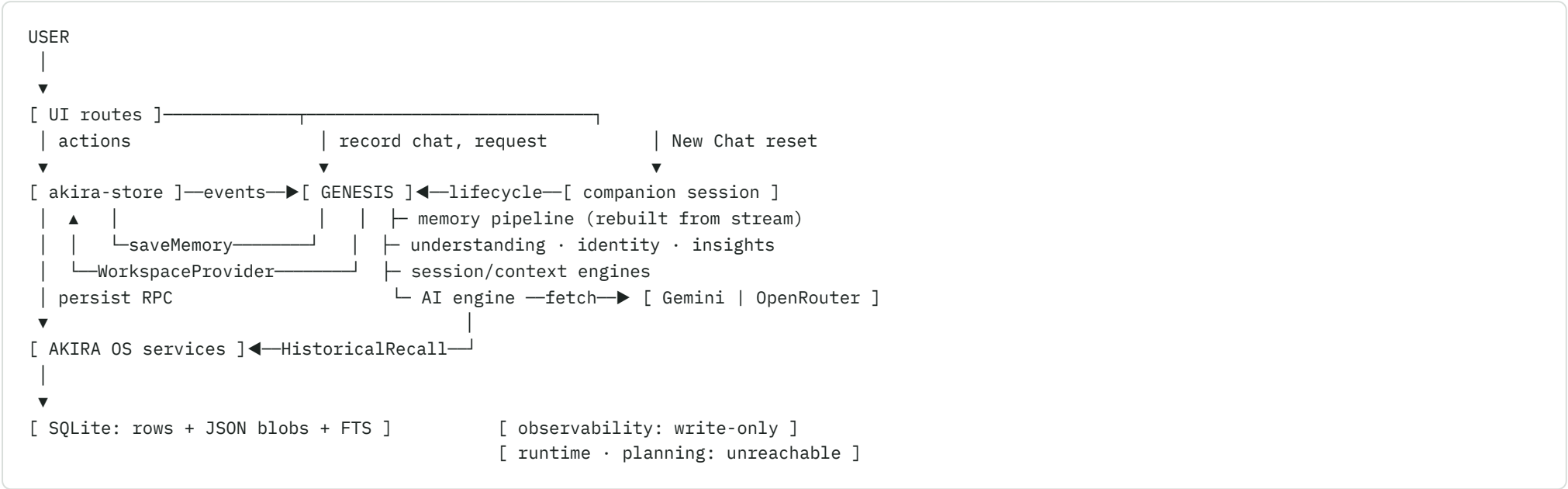
1. **identity.* accumulation.** Do these events really pile up in `genesis_memories` on each cold start and New Chat? This needs a runtime test that hydrates, replays and counts.
2. **GENESIS on the server.** Is GENESIS also composed in the server runtime during SSR? If so, the reality adapter may record into a server-side GENESIS on every RPC-republished event.
3. **Deployment target.** The nitro build preset is `cloudflare-module`, but the server uses `better-sqlite3`, the file system and `process.cwd()` to load the schema. How the desktop app is actually packaged and launched is not in the repository.
4. **Vault folder ids.** The store mints a `uid()`, but the RPC sends no id and the repository mints its own `randomUUID`. Until reload, operations on a new folder target an id the database doesn't have. The runtime effect (silent no-op, validation throw, or FK failure) is unverified.
5. **Timeline service.** Does `timelineService.initialize()` in `__root.tsx` do anything? It returns early in the browser, so the server subscriber seems to come only from the RPC.
6. **Server events before registration.** Vault events published on the server before any client RPC has registered the persistence and timeline subscribers may reach neither table.
7. **Listener order.** The memoryService listener order depends on module evaluation order. React StrictMode's double effects on the close-session / bootstrap cycle are also unverified.
8. **Batch interactions.** How a `record()` issued inside the understanding flush interacts with the batch, and whether context sees foundation goals one rebuild late.
9. **Build environment.** Whether a production build sets `VITE_GEMINI_API_KEY` or `VITE_OPENROUTER_API_KEY`, which would put a key in the bundle.
10. **Unread tables.** Who reads the `events` table in production (only unreachable analytics does), and what its retention is.
11. **Unloaded migration.** Whether `migrations/002_file_vault.sql` is ever executed. No loader was found.
12. **Reachable but unused registries.** Strategy and provider registries that are reachable but possibly never invoked: decision, reasoning, relevance, reflection, timeline renderers.
13. **Intended or not.** Whether @mentions in AKIRA's own replies are meant to create contacts. The code has no role filter.
14. **Import counts.** Fan-in and fan-out counts come from a regex import parser. They are close, not authoritative.

Documentation that conflicts with code

- The "DORMANT" header in `reality-adapter.ts` ; the adapter is live.
- The `event-service.ts` comment about the legacy bus forwarding events; that forwarding was removed.
- ADR-016..020 (runtime) and ADR-011..015 (analytics) are Accepted, but the code is unreachable.
- `06-habit-intelligence.md` describes long-term habits that can't be reached.
- CLAUDE.md points to `scripts/check-module-boundaries.mjs` ; the file is in `scripts/claude/` .
- The akira-store comment counts 29 persist operations; the code has 31.

Final architecture summary

AKIRA today



Major systems

System	Responsibility
UI routes	Pages; the conversation loop (chat.tsx); boot (__root.tsx)
akira-store	Workspace state, write-through, event emission
AKIRA OS	Domain RPCs per module

Persistence	SQLite schema, migrations, repositories, hydration
Instrumentation	Platform bus, events table, timeline
GENESIS	Cognition and context assembly
Observability	Health and telemetry (write-only)

Major engines

Engine	Role	Status
eventService + reality adapter	Intake and the durable recorder	live
memoryService	Runtime memory, reconstruction	live
stories · importance · recall	Narrative, signals, retrieval	live
understanding · identity · insights	Interpreted knowledge of the user	live
companion state · relationships · habits · resolution	Session and situational context	live
initiative · reflection · hypotheses	Proactivity, reflection, onboarding	inert
aiContextEngine + providers	Prompt assembly and model dispatch	live
planning · reasoning · decision · runtime	Objectives, reasoning, module control	unreachable

Data ownership · authority · communication · orchestration

- **Data:** the store owns the workspace and the durable stream; memoryService owns runtime memory; goals are split across three systems; conversations belong to the conversations module; configuration belongs to the provider manager.

- **Authority:** the store (state), routes (sessions, resets), the companion session (memory lifecycle), the AI engine (what the model sees). Nothing has cross-system command authority.
- **Communication:** synchronous subscriptions and direct calls; one topic-less platform bus; RPC to the server; providers as seams.
- **Orchestration:** `__root.tsx` (boot), `composition.ts` (wiring), `batch.ts` (ordering), companion session (lifecycle), AI engine (per request).

Potential VAJRA entry points (evidence only)

The per-request `aiContextEngine` ; the inert `initiativeService` ; the unreachable `runtime/` control stack; the boot lifecycle in `__root.tsx` ; the companion session's lifecycle ownership. Part 15 sets out the tradeoffs.

Biggest architectural questions before designing VAJRA

1. Should VAJRA run in the browser where cognition lives today, on the server where the data lives, or should cognition move?
2. What happens to the accepted but unreachable `runtime/` stack: adopt it, supersede it, or remove it?
3. Who owns lifecycle: VAJRA, the root page, or the companion session? Today the companion session restarts the memory layer on every New Chat.
4. What is a goal? Project arcs, foundation `IdentityGoal` and Goal understandings are three representations with no single owner, and planning has a fourth.
5. Should AKIRA ever act on its own? If yes, through what executor, and with what confirmation, given that the model's output is never parsed today?
6. Should model calls, and the API keys, stay in the browser?
7. What is the durable source of truth for cognition: an event stream replayed at every start, or stored derived state? And how should it grow beyond the caps?
8. Should system-generated events (`identity.*` , planning) share the durable stream and its caps with user events?
9. Should health and failures be surfaced and acted on, or stay write-only?
10. What is a "session": a work session, a chat conversation, or a companion session? All three exist, with different owners.
11. Does reset clear the conversation archive before history is fed to the model by default?

This audit describes the AKIRA implementation as found in the repository. It does not represent the proposed VAJRA architecture.